

GIVA - Análise Técnica Completa da Plataforma

1. GIVA = Gestão Institucional de Vagas e Avaliações

Cliente: IPIZ-360-ACADEMY (Instituto Politécnico Industrial de Zango)

Versão actual: 0.1.1 · Branch: main

CTO: Cláudio Afonso Henriques.

1.1. Stack Tecnológica || Linguagens

Linguagem | Versão | Onde foi usada | Porquê?

JavaScript (ESM) | ES2022+ | Todo o frontend - serviços, utilitários, configuração | Ecossistema React nativo; type: "module" no package.json garante módulos ES puros sem transpilação extra.

JSX | React 18 | Todos os 27 componentes e páginas em src | Sintaxe declarativa que o Vite converte via Babel/SWC em `createElement` calls; permite colocar lógica e markup no mesmo ficheiro |

CSS (puro, ~2.850 linhas) | CSS3 | style-modern.css — único ficheiro de estilos globais | Decisão deliberada: sem CSS-in-JS nem .module.css, para manter o CSS auditável, sem runtime overhead e compatível com qualquer futuro framework.

SQL (PostgreSQL) 14+ | Todos os ficheiros em docs/architecture/*.sql | Linguagem de definição e manipulação da base de dados Supabase; o PostgreSQL é o motor relacional subjacente ao Supabase.

1.2. Frameworks e Bibliotecas de Produção

React 18.3.1

- Onde: App.jsx, todas as páginas em pages, todos os componentes em components

- Porquê: Modelo de componentes com estado reactivo via hooks;

React.lazy() + <Suspense> para code splitting automático por rota; createPortal nos modais para renderizar fora da hierarquia DOM normal (e.g. JobApplicationModal, StudentProfileModal)

- Detalhe técnico: Usa useMemo, useCallback, useEffect, useRef, useState “nunca class components”. O “useOutletContext()” do React Router transporta estado global (query, showToast, t, currentDate) para todas as páginas sem prop drilling.

React Router DOM 6.28.0

- Onde: App.jsx (definição de rotas), AppShell.jsx (navegação), todas as páginas que usam useNavigate, useParams, useOutletContext, useSearchParams, useLocation.

- Porquê: Roteamento client-side (SPA); o padrão Route element={<AppShell />} com <Outlet> permite que o shell (sidebar + topbar mobile) seja partilhado por todas as páginas sem re-render.

- Detalhe técnico: future: { v7_startTransition: true, v7_relativeSplatPath: true } nos testes prepara a migração para React Router v7 sem breaking changes. Existem 16 redirects de caminhos legados (e.g. /est → /estagios, docs → /documentos)

@supabase/supabase-js 2.101.1

- Onde: supabase.js — cliente singleton; consumido por todos os 13 serviços em services

- Porquê: Abstrai completamente o PostgreSQL + Auth + Storage numa API JavaScript type-safe; evita implementar um backend REST próprio; Row Level Security (RLS) no Supabase garante isolamento de dados por utilizador sem lógica no frontend.

- Detalhe técnico crítico de segurança:

JS {

```
export const isSupabaseConfigured =
```

```
  !isTestMode && Boolean(supabaseUrl) && Boolean(supabaseAnonKey);
```

```
export const supabase = isSupabaseConfigured
```

```
  ? createClient(supabaseUrl, supabaseAnonKey) : null;
```

```
}
```

O cliente é null em modo de teste e quando as variáveis de ambiente não estão definidas — evita chamadas acidentais à produção durante os testes.

1.3. Ferramentas de Build e Desenvolvimento

Vite 5.4.10

- Onde: vite.config.js, scripts dev / build / preview: no package.json

- Porquê: Build extremamente rápido baseado em ESBuild (Go); HMR (Hot Module Replacement) instantâneo em desenvolvimento; tree-shaking agressivo na build de produção

- Detalhe técnico:

JS{

```
manualChunks: { vendor: ["react", "react-dom", "react-router-dom"] }
```

```
}
```

Separa as bibliotecas de vendor num chunk independente (vendor.js). Isto significa que quando o código da aplicação muda, o browser reusa o chunk de vendor em cache — reduz o tempo de carregamento em actualizações subsequentes.

@vitejs/plugin-react 4.3.3

- Onde: vite.config.js
- Porquê: Activa o transform JSX → JS via Babel (com Fast Refresh em dev) e SWC em prod. Sem este plugin, o Vite não consegue processar ficheiros .jsx

PostCSS 8.5.8 + @tailwindcss/postcss 4.2.2

- Onde: postcss.config.js — aplicado a todo o CSS durante o build
- Porquê: Tailwind v4 não é usado como um framework de classes utilitárias neste projecto — está presente como plugin PostCSS para garantir compatibilidade futura e autoprefixing. O CSS real é escrito manualmente em style-modern.css
- Detalhe técnico: O autoprefixer (10.4.27) adiciona automaticamente prefixos -webkit- , -moz- etc. para compatibilidade cross-browser, especificamente para backdrop-filter (usado no topbar mobile e backdrop do sidebar).

1.4. Ferramentas de Teste

Vitest 2.1.8

- Onde: app.routes.test.jsx (29 testes), setup.js
- Porquê: Compatível 100% com a API do Jest mas vive dentro do ecossistema Vite — usa o mesmo bundler, mesma configuração, mesma resolução de módulos. Sem Vitest, testar JSX com Vite exigiria configuração separada do Jest
- Configuração:

JS{

```
test: { environment: "jsdom", setupFiles: "./src/test/setup.js", globals: true }
```

jsdom simula o DOM do browser em Node.js. globals: true disponibiliza describe/it/expect sem imports.

}

@testing-library/react 16.1.0

- Onde: app.routes.test.jsx — render, screen, waitFor, fireEvent, within.

- Porquê: Testa como o utilizador interage com a UI, não os detalhes de implementação. `getByRole("navigation", { name: /main menu/i })` verifica acessibilidade real, não selectors CSS frágeis.

@testing-library/jest-dom 6.6.3

- Onde: `setup.js` — importado via `import "@testing-library/jest-dom/vitest"`
- Porquê: Adiciona matchers semânticos como `toBeInTheDocument()`, `toHaveClass()`, `toBeDisabled()` — mais legíveis que `expect(el).not.toBe(null)`.

jsdom 25.0.1

- Onde: Runtime de teste (configurado pelo Vitest)
- Porquê: Simula `window`, `document`, `localStorage`, `matchMedia` em ambiente Node.js puro.

1.5. Plataforma de Infraestrutura

Supabase (BaaS — Backend as a Service)

- Serviços usados:
 - **Auth**: JWT tokens, sessões, `signInWithPassword`, `onAuthStateChange`.
 - **Database** (PostgreSQL): 13 tabelas principais com RLS
 - **Storage**: Bucket `student-profiles` para fotos de perfil de alunos
 - **Realtime**: Não usado actualmente (potencial futuro)
- **Detalhe técnico** — RLS (Row Level Security):

As políticas **SQL** garantem que um aluno só lê os seus próprios dados, um coordenador só acede à sua ``area_id``, sem qualquer validação no frontend:

SQL:

```
create policy "Users can upload their own photo"
on storage.objects for insert to authenticated
with check (auth.uid() = (storage.foldername(name))[1]::uuid);
```

Vercel (Hosting)

- Onde: `vercel.json`
- Porquê: Deploy automático de SPAs com suporte nativo ao Vite; CDN global; HTTPS automático

Detalhe técnico crítico:

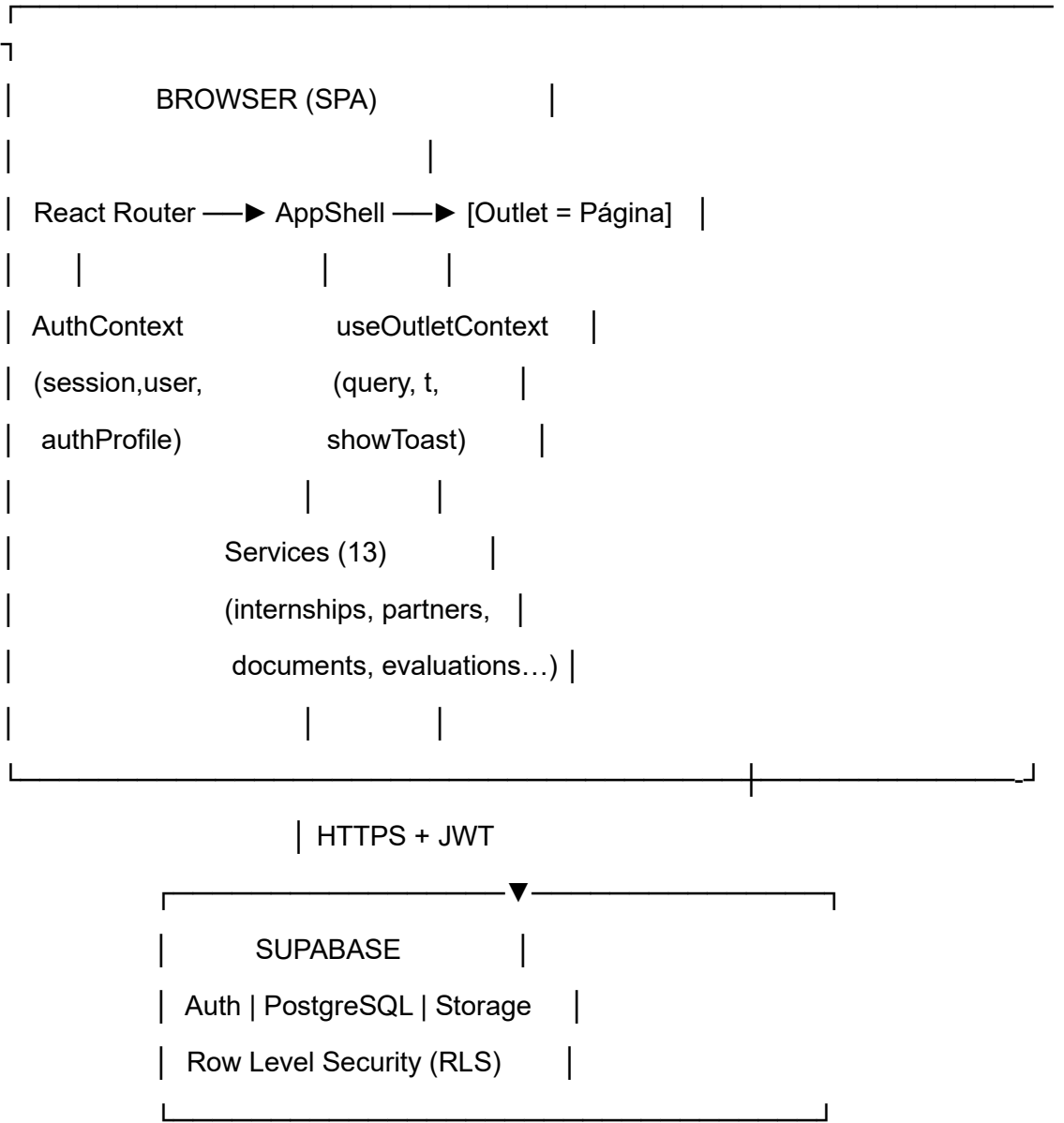
```
json

"rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
```

Todas as rotas são redirecionadas para index.html — necessário para que o React Router funcione correctamente ao navegar directamente para URLs como /estagios/detalhe?turma=11-TI-A.

2. Arquitectura da Plataforma

2.1 Padrão Geral



2.2 Padrão de Serviços

Todos os 13 serviços seguem o mesmo contrato:

1 - Guard de disponibilidade

```
export function canUseXxxApi() {  
  return typeof supabase !== "undefined" && supabase !== null;  
}
```

2 - Operações async que lançam/retornam null em caso de erro

```
export async function listXxx() {  
  if (!canUseXxxApi()) return [];  
  const { data, error } = await supabase.from("xxx").select("*");  
  if (error) { console.error(...); return []; }  
  return data;  
}
```

Esta abordagem tem um propósito de segurança: nunca expõe erros brutos do Supabase ao utilizador final, e permite que a aplicação funcione sem backend configurado (modo demo/dev).

2.3 Separação de Responsabilidades

| Camada | Localização | Responsabilidade |

Routing + Auth Guard | App.jsx, RequireAuth.jsx | Controlo de acesso a rotas; redirect para /login se não autenticado |

| Shell/Layout | AppShell.jsx | Sidebar, topbar mobile, toast system, pesquisa global, context de outlet |

| Context | AuthContext.jsx | Estado de sessão reactivo; expõe user, authProfile, isAuthenticated |

| Páginas | pages (16 páginas) | Orquestram serviços, gerem estado local, passam dados a componentes |

| Componentes | components (13 componentes) | UI reutilizável; recebem dados via props; não chamam serviços directamente (excepto ExpandedStudentProfile e CompanyProgressTimeline) |

| Serviços | services (13 serviços) | Única camada que interage com o Supabase; normalizam dados da BD |

| Utilitários | utils | i18n.js (3 idiomas × ~350 chaves cada), search.js (matchesSearch) |

| Base de dados | Supabase/PostgreSQL | Validação de tipos, RLS, triggers updated_at, índices de performance |

3. Apresentação dos Dados

3.1 Fluxo de Dados por Módulo

Dashboard (Coordenador)

- 5 chamadas paralelas via Promise.all: estagios, parceiros, documentos, notificações, candidaturas do aluno.
- KPIs calculados no cliente com .filter() e .length — sem views SQL dedicadas
- Dados agrupados por useMemo para evitar recálculos desnecessários

Estagios

- listInternships() → tabela internships → normalização com normalizeInternshipRow() (sanitização de XSS via toSafeText() e validação de URLs)
- Agrupados por anoLetivo | turma em memória via Map.
- Filtros em memória com useMemo (status, ano lectivo, turma, texto livre)
- Paginação por turma em memória com pageSize configurável (3/5/10) persistido em localStorage.

Estatísticas

- fetchStatisticsMetrics() agrega 3 tabelas: internships, job_applications, evaluations
- Cálculo no cliente: $\text{completion} = ((\text{active} + \text{completed}) / \text{total}) * 100$ - Fallbacks hardcoded (88%, 71%, 6%, 92%) enquanto Supabase não está configurado.

Documentos

- CRUD completo: listDocuments, createDocument, updateDocument, deleteDocument, updateDocumentStatus, uploadDocumentFile.
- Upload vai para Storage Supabase com URL pública gerada por getPublicUrl().

3.2 Normalização e Segurança de Dados

Cada serviço normaliza os dados antes de os entregar às páginas:

JS

internshipsService.js — previne XSS e injection

```
function toSafeText(value) {  
    return String(value ?? "")  
        .replace(/[ 0000- 001F 007F]/g, "") // control chars  
        .replace(/<|>/g, "")                // < > para prevenir HTML injection  
}
```

```

        .trim();
    }
    function toSafeImageUrl(value) {
        if (raw.startsWith("https://") || raw.startsWith("http://")) {
            return encodeURI(raw); // encode URLs externas
        }
        // data: URIs e caminhos relativos passam sem encode
    }

```

3.3 Sistema i18n

- 3 idiomas: pt-BR, pt-PT, en.
- 350 chaves por idioma no objecto “TRANSLATIONS”
- Função createTranslator(language), que retorna t(key) com fallback para key se não existir.
- Persistência: localStorage("giva.preferences") guarda idioma, densidade visual e preferência de notificações.
- Aplicação: t é passado via **useOutletContext** para todas as páginas e componentes — zero imports de i18n nas páginas.

4. Dimensionamento

4.1 Código

| Métricas | Valor |

| Páginas React | 16 (DashboardPage, InternshipsPage, ClassesPage, ClassDetailPage, PartnersPage, StatisticsPage, DocumentsPage, NotificationsPage, StudentPage, StudentProfilePage, StudentProgressPage, TrainingAreasPage, EvaluationsPageEnhanced, CompanyDashboardPage, LoginPage + 4 páginas de Settings) |

| Componentes React | 13 (AppShell, ClassRegisterModal, CompanyProgressTimeline, DataTable, DocumentSubmitModal, ExpandedStudentProfile, JobApplicationModal, PageHeader, PanelSection, PartnerRegisterModal, RequireAuth, StudentProfileModal, TrainingAreaCard) |

| Serviços | 13 ficheiros em services |

| CSS global | ~2.850 linhas, 1 ficheiro |

| i18n | ~1.050 chaves totais (3 idiomas × 350) |

| Scripts de manutenção | 5 em scripts (audit-i18n, add-missing-i18n, provision-users, fix-i18n PS1) |

| Testes | 29 testes num único ficheiro de integração |

4.2 Build de Produção

| Chunk | Tamanho (gzip) | Conteúdo |

| vendor.js | ~144 KB | React + React DOM + React Router |

| index.js | ~35-40 KB | Toda a aplicação (páginas lazy-loaded) |

| style-modern.css | ~37 KB | CSS global completo |

| Total inicial | ~180 KB | Carregado no primeiro acesso |

| Chunks lazy | ~5-15 KB cada | Páginas individuais carregadas a pedido |

4.3 Base de Dados (PostgreSQL via Supabase)

Tabelas principais:

| Tabela | Campos-chave | Relações |

| training_area | id, code, name, color_hex, icon_name, display_order | ← courses, students |

| courses | id, training_area_id, code, name | → training_area |

| students | id, training_area_id, course_id, full_name, email, phone_number, skills[], languages jsonb, profile_photo_url | → training_area, courses |

| student_portfolio | id, student_id, type, title, description, tags | → students (cascade delete) |

| partners | id, empresa, nif, setor, areas[], vagas, created_by | ← job_applications |

| job_applications | id, student_id, partner_id, status (PENDING/ACCEPTED/REJECTED), applied_at | → students, partners |

| company_progress | id, student_id, partner_id, progression_stage, interview_date, contract_type | → students, partners |

| evaluations | id, training_area_id, student_id, evaluation_type (INDIVIDUAL/GROUP), score, is_final | → training_area, students |

| internships | id, aluno, turma, ano_letivo, empresa, nota, status | standalone |

| documents | id, titulo, tipo, versao, arquivo_url, estado | standalone |

| notifications | id, title, prioridade (high/medium), lida | standalone |

Extras de infraestrutura:

- Trigger update_updated_at_column() em 8 tabelas

- Índices em `training_area(code)`, `training_area(is_active, display_order)`, audit events
- RPC Function `increment_vagas_preenchidas(partner_id)` — atómicamente incrementa contador de vagas.

4.4 Permissões (RBAC)

| Role | Acesso |

| SUPER_ADMIN | Global — todas as áreas, todos os dados |

| ADMIN_1 | Gatekeeper — aprova acções sensíveis, acede ao painel empresa e gestão de candidaturas |

| ADMIN_2 | Coordenador por área — opera apenas na sua `area\_id` |

| STUDENT | Escopo próprio — candidaturas, perfil pessoal, progresso individual |

| PROFESSOR | Criar avaliações individuais e de grupo |

5. Etapas de Construção

Fase 0 — Baseline HTML/CSS (pré-React)

- Prototipagem em HTML+CSS puro no `index.html`
- Definição dos tokens CSS: `--primary`, `--bg`, `--bg-elevated`, `--border`, `--shadow-`, `--radius-`, `--space-`, `--motion-`
- Aprovação visual antes de avançar para React

5.1 Fase 1 — Setup React + Roteamento

- Migração de HTML puro para SPA React + Vite
- Implementação do `App.jsx` com todas as rotas
- `RequireAuth.jsx` — guarda de autenticação
- `AuthContext.jsx` + `authService.js` — sessão via Supabase Auth

5.2. Fase 2 — Shell e Navegação

- `AppShell.jsx` — múltiplas iterações:
 - Flyout grid nav → lista vertical → sidebar pushing content → sidebar permanente colapsada
 - Estado final: `4.5rem` colapsado (só ícones), 260px expandido ao hover
 - Labels com `opacity: 0` → opacity: 1 com delay após expansão
 - Topbar mobile com botão hamburger + logo IPIZ + backdrop

5.3 Fase 3 — Módulos de Dados

- 13 serviços implementados com padrão `canUseXxxApi()` + `guard` + `normalize`
- Módulos: estagios, parceiros, documentos, notificações, avaliações, turmas, perfil do aluno, portfólio, candidaturas, progresso na empresa, estatísticas, notas de aluno

5.4 Fase 4 — Páginas e Componentes

- 16 páginas com `fetch` real via serviços
- 13 componentes reutilizáveis
- Sistema de modais com `createPortal` (fora da hierarquia DOM)
- ``DataTable` genérico com colunas configuráveis

5.5 Fase 5 — i18n e Acessibilidade

- 3 idiomas completos (~1.050 chaves)
- `aria-label`, `role="dialog"`, `aria-modal`, `aria-current="page"` em toda a navegação
- Testes de acessibilidade via `getByRole` no Vitest.

Fase 6 — Testes e Qualidade

- 29 testes de integração cobrindo todas as rotas, redirecionamentos legados, login em PT/EN, navegação
- Scripts de auditoria i18n (`audit-i18n.mjs`, `add-missing-i18n.mjs`)
- Validação OWASP: sanitização de inputs, `encodeURIComponent` em URLs externas, sem `dangerouslySetInnerHTML`

Fase 7 — CSS Refinamento e Responsividade

- Breakpoints: ``1280px` (desktop largo), `980px` (tablet), `640px` (mobile), `480px` e `390px` (mobile pequeno).
- CSS tokens para todas as propriedades de espaçamento, sombras, raios
- Remoção de valores hardcoded, standardização com `var(--*)`.

6. Análise dos Resultados (Estado actual)

O que funciona completamente

-  Autenticação (Supabase Auth) com sessão persistente
-  Sidebar adaptativa (colapsado/expandido/mobile)
-  i18n em 3 idiomas com persistência
-  CRUD completo de Parceiros, Documentos, Turmas
-  Sistema de candidaturas aluno↔empresa (submit, accept, reject)
-  Timeline de progresso na empresa (6 fases)
-  Perfil expandido do aluno com portfólio
-  Avaliações individuais e em grupo com exportação CSV
-  Estatísticas em tempo real calculadas da BD
-  29/29 testes passam; build limpo sem warnings

O que está parcialmente implementado

-  Approval Flow (arquitectura documentada, tabelas criadas, fluxo UI não implementado).
-  Auditoria (audit_event na BD, sem UI de consulta).
-  Notificações em tempo real (Supabase Realtime configurável mas não activado).
-  SettingsSecurityPage — alteração de password funciona mas 2FA e timeout são apenas UI sem lógica.

Débito técnico identificado

- internshipsService.js usa dados de teste hardcoded (TEST_INTERNSHIPS) em modo de teste — não é um mock de Supabase real.
- CompanyDashboardPage e ExpandedStudentProfile chamam serviços directamente (quebrando levemente a separação página↔componente)
- EvaluationsPageEnhanced aceita IDs de alunos como CSV em texto livre — deveria ter um picker.

7. Análise e Custo do Projecto

7.1 Custos de Infraestrutura (Mensais — Estimativa)

Serviço	Plano	Custo/mês	Limites incluídos
Supabase	Free Tier	\$0	500 MB DB, 1 GB Storage, 50 MB ficheiros, 500 MB bandwidth, 50.000 auth users
Supabase	Pro (quando escalar)	\$25	8 GB DB, 100 GB Storage, 250 GB bandwidth
Vercel	Hobby (deploy actual)	\$0	100 GB bandwidth, builds ilimitados
Vercel	Pro (produção)	\$20	Builds concorrentes, analytics, password protection
Domínio (ipiz.ao ou similar)	—	~\$40-50/ano	—
Total dev	—	\$0/mês	Dentro dos free tiers
Total produção	—	\$45-55/mês	Pro Supabase + Pro Vercel

7.2 Custos de Desenvolvimento (Estimativa de Mercado — Angola/Portugal/EUA)

Componente	Horas estimadas	Custo (dev júnior ~\$15/h)	Custo (dev sénior ~\$50/h)
Arquitectura + BD (SQL, RLS, docs)	20h	\$300	\$1.000
Setup Vite + React + Auth	8h	\$120	\$400
AppShell + i18n + routing	16h	\$240	\$800
13 Serviços Supabase	20h	\$300	\$1.000
16 Páginas	40h	\$600	\$2.000
13 Componentes	30h	\$450	\$1.500
CSS global (~2.850 linhas)	24h	\$360	\$1.200
29 Testes + scripts QA	10h	\$150	\$500
Total	~168h	~\$2.520	~\$8.400

7.3 Custos de Operação Anual (Produção)

| Item | Custo anual |

| Infraestrutura (Supabase Pro + Vercel Pro) | ~\$540 |

| Domínio | ~\$20 |

| Manutenção técnica (5h/mês × \$15-50/h) | \$900–\$3.000 |

| Total anual | ~\$1.460–\$3.560 |

8. Decisões de Arquitectura — Porquê Cada Escolha

| Decisão | Alternativas rejeitadas | Justificação |

| CSS puro em 1 ficheiro | Tailwind classes, CSS Modules, Styled Components | Zero runtime overhead; auditável por designers sem React; fácil de entregar a terceiros |

| Sem Redux/Zustand | Redux Toolkit, Zustand, Jotai | A aplicação tem estado maioritariamente server-derived; useOutletContext + AuthContext cobrem os casos sem complexidade adicional |

| Supabase como BaaS | REST API própria em Node.js/Express | Elimina backend, infraestrutura, SSL, autenticação e RBAC custom — meses de desenvolvimento |

| Vitest em vez de Jest | Jest + Babel | Mesma API, mas integrado no Vite — zero configuração separada; mais rápido |

| i18n próprio (sem react-i18next) | react-i18next, linguiJS | O sistema é simples, sem plurais complexos nem ICU — adicionar uma dependência de 50KB seria exagero |

| createPortal nos modais | Modais inline no DOM | Evita problemas de z-index e overflow:hidden nos ancestrais; modais renderizam directamente em document.body |

| Lazy loading em todas as páginas | Import estático | O bundle inicial fica em ~180KB gzipped; páginas raramente visitadas (config, empresa) não bloqueiam o carregamento inicial.

Obs: Até ao momento em custos para a construção do projecto funcional gastou-se mais de 45 mil kwanzas distribuidos em: Internet, Mentorias com Profissionais na Área na Pessoa de Manuel Dembo, Fabio Junik, Baptista Chuma e Pedro Muteka.

Que pessoalmente estão organizando formas de preparar o grupo como co-tutores